

# Singable Lyrics Translation with IPA-Augmented LLM Agents

Anonymous ACL submission

## Abstract

Translating song lyrics while preserving singability requires satisfying musical constraints—syllable counts, rhyme schemes, and syllable patterns—that standard machine translation ignores. We present a framework that equips a large language model with IPA-based phonetic analysis tools within a ReAct-style agent loop. The agent extracts constraints from source lyrics, then translates through a multi-phase process, calling tools to iteratively verify and refine each constraint. We formalize the three musical constraints, describe the IPA tool suite and agent architecture, and propose three evaluation metrics: Syllable Error Rate (SER), Syllable Count Relative Error (SCRE), and Adjusted Rand Index (ARI).

## 1 Introduction

A translated song lyric must conform to the musical structure of the original so that it remains *singable* to the same melody (Low, 2005). This requires satisfying hard structural constraints: syllable counts must match melodic phrases, rhyme schemes should be preserved, and word-level syllable distributions should mirror the original phrasing. Despite theoretical grounding (Low, 2005; Franzon, 2008), computational approaches remain scarce. Neural MT systems have no mechanism for phonetic constraints, and constrained decoding methods (Hokamp and Liu, 2017; Post and Vilar, 2018) address lexical but not phonetic constraints. A fundamental obstacle is that *counting syllables, detecting rhyme, and analyzing phonetic patterns are not tasks that LLMs perform reliably through text generation alone* (Gao et al., 2023).

We address this by equipping an LLM with IPA-based phonetic tools within a ReAct-style agent loop (Yao et al., 2023), enabling iterative, constraint-aware translation refinement. Our framework is open-source and available as a PyPI pack-

age for reproducibility and practical use.<sup>1</sup>

Our contributions are:

1. A formalization of three musical constraints—syllable counts, rhyme schemes, and syllable patterns—and IPA-based methods for their extraction and verification (§3).
2. An IPA-augmented tool suite and multi-phase agent loop architecture for constraint-preserving lyrics translation (§4).
3. Three automatic evaluation metrics—SER, SCRE, and ARI—designed to measure structural constraint preservation in singable translation (§5).

## 2 Related Work

### 2.1 Song Translation

Low (2005) proposed treating song translation as a pentathlon balancing singability, sense, naturalness, rhythm, and rhyme, observing that rhythm is often the hardest constraint. Franzon (2008) identified five translation strategies ranging from adapting lyrics to rewriting both lyrics and melody. Low (2022) provided an updated theoretical account reinforcing singability as the guiding criterion.

On the computational side, Ou et al. (2023a) formalized singable lyric translation as a constrained sequence-to-sequence problem, fine-tuning a neural MT model with length, rhyme, and word boundary constraints for English–Chinese translation. Guo et al. (2022) addressed automatic song translation for tonal languages, introducing tonal constraints beyond syllable matching, and Ghazvininejad et al. (2018) tackled neural poetry translation with rhythm and rhyme constraints using constrained decoding. Ye et al. (2024) proposed

<sup>1</sup>Code and package available at <https://github.com/anonymous> (anonymized for review).

quality-aware musical lyrics translation that explicitly models musical features during generation. Ren et al. (2025) introduced a curriculum reinforcement learning framework for controllable lyric translation, balancing multiple constraints through difficulty-aware training. Zhao et al. (2025) addressed melody-constrained lyrics editing, and Ou et al. (2023b) enforced structural wording and formatting constraints in melody-to-lyric generation. Cho et al. (2025) introduced MAVL, a multilingual audio-video lyrics dataset, and Kim et al. (2023) proposed a computational evaluation framework for singable lyric translation. Beyond translation, Chen and Teufel (2024) used scansion to generate Mandarin lyrics matching melodies, Liang et al. (2024) improved singability of AI-generated lyrics through prosody explanations, Yu et al. (2025) modeled lyric-melody alignment for generation, and Yoo et al. (2025) demonstrated that singability challenges extend to multimodal settings such as sign language. These approaches rely on specialized model fine-tuning, constrained decoding, or reinforcement learning; our work instead uses a general-purpose LLM augmented with IPA verification tools in an agent loop, requiring no task-specific training.

## 2.2 Constrained Text Generation

Hokamp and Liu (2017) and Post and Vilar (2018) introduced lexically constrained decoding via Grid Beam Search and Dynamic Beam Allocation. In computational poetry, Ghazvininejad et al. (2016) used finite-state machinery for meter and rhyme, Hopkins and Kiela (2017) generated rhythmic verse with neural networks, and Lau et al. (2018) jointly modeled poetic language, meter, and rhyme. These approaches enforce constraints at the decoding level or learn them implicitly; ours uses an agent loop where the LLM generates freely but verifies via external tools.

## 2.3 Tool-Augmented Language Models

Recent work has shown that LLMs can be made substantially more capable by giving them access to external tools. Schick et al. (2023) trained language models to autonomously decide when to call tools such as calculators and search engines. ReAct (Yao et al., 2023) introduced a paradigm where LLMs alternate between generating reasoning traces and performing actions via external tools, improving performance on tasks requiring grounded, multi-step decision-making. PAL (Gao et al., 2023)

demonstrated that offloading computational steps to program execution significantly improves accuracy on reasoning tasks, a finding directly relevant to our use case: syllable counting is a computational task that LLMs perform unreliably but that IPA-based tools handle deterministically. Chain-of-thought prompting (Wei et al., 2022) provides a complementary mechanism for structured reasoning that we combine with tool use in our agent loop. In machine translation, Jiao et al. (2023) demonstrated that LLMs achieve strong translation quality, but their evaluation did not consider musical or structural constraints.

## 2.4 Phonetic Resources

Our tool suite builds on Phonemizer (Bernard and Titeux, 2021) for text-to-IPA conversion and Pan-Phon (Mortensen et al., 2016) for phonetic distance computation; both are described in §4.

## 3 Musical Constraints in Lyrics Translation

A singable translation must satisfy three structural constraints, each arising from a different aspect of the relationship between lyrics and melody. Below we describe each constraint, formalize it, and explain how we extract and verify it using IPA-based analysis.

### 3.1 Syllable Count Constraint

In sung performance, each syllable maps to one or more musical notes. When a melody assigns three notes to a phrase, the lyric must contain exactly three syllables—fewer leave notes unmatched, while extra syllables force cramming that disrupts the rhythmic feel. This makes syllable count the most rigid of the three constraints: rhyme and phrasing mismatches may be tolerable in some styles, but syllable count mismatches are almost always audible. Formally, given source lyrics  $L_s = (l_1, \dots, l_n)$  in language  $\lambda_s$ , the constraint requires that translated lyrics  $L_t = (l'_1, \dots, l'_n)$  in target language  $\lambda_t$  satisfy  $\text{syll}(l'_i, \lambda_t) = \text{syll}(l_i, \lambda_s)$  for all lines  $i$ .

Syllable counting is language-dependent. For Chinese, each character constitutes exactly one syllable, so counting is trivial. For other languages, we convert text to IPA via Phonemizer (Bernard and Titeux, 2021) and count *vowel nuclei*—the vocalic centers of syllables:

$$\text{syll}(l_i, \lambda_s) = |\text{nuclei}(\text{IPA}(l_i, \lambda_s))| \quad (1)$$

where  $\text{nuclei}(\cdot)$  matches a predefined set of IPA vowel symbols and diphthongs (e.g., a, i, u, ə, ai, ei, ou), with adjacent vowels forming diphthongs counted as single nuclei. For example, “*Let it go*” yields IPA /lət it ɡəʊ/ with three vowel nuclei [ɛ, ɪ, əʊ], giving a count of 3. The resulting sequence  $s = (s_1, \dots, s_n)$  serves as the hard constraint for translation.

### 3.2 Rhyme Scheme Constraint

Rhyme provides structural cohesion in lyrics, creating sonic expectations that are satisfying when met and jarring when violated. A song with an *AABB* scheme feels fundamentally different from one with *ABAB*; preserving the rhyme scheme ensures the translation maintains the same pattern of sonic echoes as the original. Formally, a rhyme scheme is a labeling  $R = (r_1, \dots, r_n)$  where  $r_i \in \{A, B, C, \dots\}$  assigns each line to a rhyme class, with  $r_i = r_j$  if and only if lines  $i$  and  $j$  rhyme. The constraint requires that the translation’s scheme  $R_t$  match the source scheme  $R_s$ .

We detect rhyme by comparing phonetic endings rather than orthographic forms. For each line, we extract the *rhyme ending*: the IPA substring from the last vowel nucleus to the end of the transcription. For Chinese, we extract the pinyin final of the last character instead. The scheme is constructed by labeling the first line *A*, and assigning each subsequent line the label of any earlier line it rhymes with, or a new label otherwise. Consider the following example:

*The stars shine bright with silver light* → /laɪt/ → ending: /aɪt/  
*And covers all in purest white* → /waɪt/ → ending: /aɪt/  
*I stand and watch the world below* → /bɪləʊ/ → ending: /əʊ/  
*As gentle winds begin to blow* → /bləʊ/ → ending: /əʊ/

Lines 1–2 share ending /aɪt/ (class *A*) and lines 3–4 share /əʊ/ (class *B*), yielding *AABB*. This IPA-based approach correctly identifies rhymes that orthographic comparison would miss (e.g., “weight” and “late”) and rejects false rhymes based on spelling alone (e.g., “cough” and “through”).

### 3.3 Syllable Pattern Constraint

Even when two lines share the same total syllable count, different word-level distributions create different rhythmic feels. Consider two eight-syllable lines: “un-for-get-ta-ble mel-o-dy” with pattern [5, 3] versus “I can hear the mu-sic play”

with pattern [1, 1, 1, 1, 2, 2]. The first has two long words producing a flowing feel, while the second has many short words producing a staccato rhythm. Word boundaries interact with note groupings and breaths in sung performance, so preserving the syllable pattern ensures the translation fits the melody at the phrasing level, not just the syllable level. Formally, the syllable pattern of a line  $l_i$  is a vector  $\mathbf{p}_i = (p_{i,1}, \dots, p_{i,k_i})$  recording the syllable count of each word. The constraint asks that the translation’s pattern  $\mathbf{p}'_i$  be similar to  $\mathbf{p}_i$  while maintaining the same total:  $\sum_j p'_{i,j} = \sum_j p_{i,j} = s_i$ .

To extract patterns, we apply language-specific word segmentation—space-based tokenization for English and other space-delimited languages, and a neural tokenizer for Chinese—then compute each word’s syllable count via Equation 1. We measure pattern similarity using a normalized alignment score. Given patterns of potentially different lengths, we pad the shorter one with zeros and compute:

$$\text{sim}(\mathbf{p}, \mathbf{p}') = 1 - \frac{\sum_{j=1}^m |p_j - p'_j|}{\sum_{j=1}^m \max(p_j, p'_j)} \quad (2)$$

where  $m = \max(|\mathbf{p}|, |\mathbf{p}'|)$ . A score of 1.0 indicates an exact match; we treat  $\geq 0.8$  as acceptable. For example, “Let it go” has pattern [1, 1, 1]. A Chinese translation tokenized as two words with pattern [2, 1] yields  $\text{sim}([1, 1, 1], [2, 1, 0]) = 0.5$  (poor alignment), whereas a translation tokenized as three single-character words gives pattern [1, 1, 1] with  $\text{sim} = 1.0$ .

## 4 IPA-Augmented Agent Framework

We now describe the agent architecture that uses IPA tools to produce constraint-preserving translations. Figure 1 provides an overview of the complete pipeline.

### 4.1 IPA as Universal Phonetic Interface

The International Phonetic Alphabet provides a standardized, language-independent representation of speech sounds, making it an ideal intermediary for cross-lingual phonetic analysis: regardless of the source or target language, all phonetic operations—syllable counting, rhyme comparison, pattern analysis—can be performed on IPA transcriptions using the same algorithms. Our system uses Phonemizer (Bernard and Titeux, 2021) as the text-to-IPA backend, wrapping the eSpeak NG

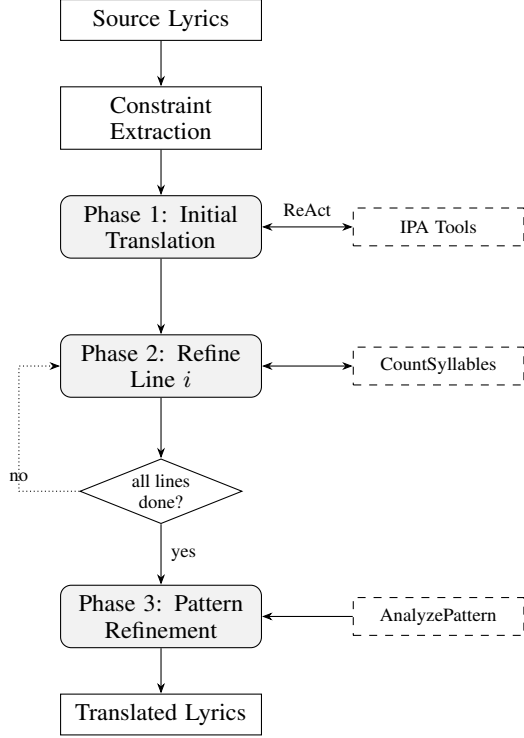


Figure 1: Agent framework overview. Phase 1 translates all lines using an internal ReAct loop with all IPA tools. Phase 2 refines each line sequentially (up to 10 attempts per line); the dotted arrow shows the graph-level loop that iterates through lines. Phase 3 is a single pass that uses ANALYZEPATTERN output to guide pattern refinement without a retry loop.

speech synthesizer with support for over 100 languages. Language codes are normalized internally (e.g., zh, zh-cn, cmn all map to Mandarin Chinese) to provide a uniform interface. For phonetic distance computation, we use PanPhon (Mortensen et al., 2016), which maps each IPA segment to a vector of articulatory features, enabling fine-grained similarity measurement.

## 4.2 Tool Suite

The agent has access to four core IPA-based tools (Table 1), each wrapping a deterministic function that provides capabilities the LLM cannot reliably perform through text generation alone (Gao et al., 2023).

COUNTSYLLABLES is the most frequently called tool: given a text string and language code, it converts to IPA and returns an integer syllable count (§3.1). TEXTTOIPA exposes the raw IPA transcription, allowing the agent to inspect phonetic realizations and reason about which syllables to modify—a form of chain-of-thought reasoning (Wei et al., 2022) grounded in phonetic observa-

| Tool           | Description                                                           |
|----------------|-----------------------------------------------------------------------|
| COUNTSYLLABLES | Count syllables via IPA vowel nuclei (or character count for Chinese) |
| TEXTTOIPA      | Convert text to IPA transcription via eSpeak NG                       |
| DETECTRHYME    | Detect the rhyme scheme of a set of lines by comparing IPA endings    |
| ANALYZEPATTERN | Compare target vs. current syllable patterns with structured feedback |

Table 1: Core IPA-based tools available to the agent. All tools accept a language parameter for language-specific processing.

| Line | Text                       | Syll. | Pattern       |
|------|----------------------------|-------|---------------|
| 1    | The snow glows white       | 4     | [1,1,1,1]     |
| 2    | On the mountain tonight    | 6     | [1,1,2,2]     |
| 3    | Not a footprint to be seen | 7     | [1,1,2,1,1,1] |
| 4    | A kingdom of isolation     | 8     | [1,2,1,4]     |

**Rhyme scheme:** *ABCB* (*tonight/seen* do not rhyme; detected via IPA ending comparison)

Table 2: Example constraint extraction from English source lyrics. Syllable counts are computed via IPA vowel nuclei; patterns via space-based word segmentation.

tion. DETECTRHYME analyzes the IPA endings of a set of lines and returns the rhyme scheme (e.g., “AABB”), ensuring judgments are phonetic rather than orthographic. ANALYZEPATTERN compares a target syllable pattern against the current one, returning the similarity score (Equation 2), per-word differences, and natural-language suggestions for improvement. All tools are registered as callable functions via modern LLM function-calling APIs.

## 4.3 Constraint Extraction

Before translation begins, the system extracts all three constraints from the source lyrics  $L_s$  in language  $\lambda_s$ : (1) **syllable counts**  $\mathbf{s} = (s_1, \dots, s_n)$  via COUNTSYLLABLES on each line, (2) **rhyme scheme**  $R_s$  via DETECTRHYME on all lines, and (3) **syllable patterns**  $\mathbf{P}_s = (\mathbf{p}_1, \dots, \mathbf{p}_n)$  via ANALYZEPATTERN. These constraints are passed to the agent as part of the translation prompt, providing explicit targets for each line. Table 2 shows an example.

## 4.4 Multi-Phase Agent Loop

Translation proceeds through three phases, implemented as a directed acyclic graph where each node represents a processing step and conditional edges



encode control flow. The design reflects the priority ordering identified by Low (2005): semantic content and rhythm (syllable counts) take precedence, followed by phrasing (patterns), with rhyme considered throughout but given lower priority.

**Phase 1: Initial Translation with Tool-Guided Reasoning.** The agent receives a structured prompt containing the source lyrics, target language, and all extracted constraints. It generates an initial translation of all lines, following a ReAct loop (Yao et al., 2023): the LLM generates reasoning about how to satisfy constraints, calls tools to verify its output, observes the results, and adjusts within the same generation turn. The prompt instructs the agent to consider all three constraints simultaneously, producing a “best-effort” initial translation.

A typical interaction proceeds as: the agent reasons about the target syllable count for a line, drafts a translation, calls COUNTSYLLABLES to verify, observes the result, and adjusts if needed—continuing until all lines are translated.

**Phase 2: Line-by-Line Syllable Refinement.** The initial translation rarely satisfies all syllable constraints exactly. In this phase, the agent processes each line individually, computing the actual syllable count and comparing it to the target. If there is a mismatch, the agent receives explicit feedback (e.g., “Line  $i$  has  $s'_i$  syllables but the target is  $s_i$ ; please revise while preserving meaning”) and generates a revised translation. The system re-checks via COUNTSYLLABLES, continuing for up to 10 iterations; if no exact match is found, the closest attempt is retained. This phase is critical because even a single syllable mismatch can make a line unsingable.

**Phase 3: Syllable Pattern Refinement.** After syllable counts are matched, the agent refines word-level syllable distributions. For each line, ANALYZEPATTERN compares the current pattern against the target and returns structured feedback including the similarity score, per-word differences, and suggestions (e.g., “split the first word into shorter components”). For each mismatched line, the agent generates a revised translation; the revision is accepted only if it improves pattern similarity while maintaining the correct total syllable count, and is otherwise reverted. Lines whose patterns already match the target are left unchanged.

**Priority and termination.** The three-phase design addresses constraints in order of importance: syllable counts first (hardest constraint), then syllable patterns. Rhyme is monitored throughout Phases 1 and 2 via DETECTRHYME, but is not given a dedicated refinement phase because rhyme violations are generally less disruptive to singability than syllable mismatches (Low, 2005). The process terminates after Phase 3 with a final validation step.

#### 4.5 Worked Example

To illustrate, consider translating “*Not a footprint to be seen*” (7 syllables, pattern [1, 1, 2, 1, 1, 1]) into Chinese. In Phase 1, the agent produces a 7-character translation; syllable counting confirms the match, so Phase 2 skips this line. In Phase 3, pattern analysis reveals the Chinese tokenization yields [3, 2, 2]—similarity 0.38 against the target—and suggests splitting the first word. The agent revises to a translation with single-character tokenization, yielding [1, 1, 1, 1, 1, 1, 1] and similarity 0.71: an improvement, though not exact. This illustrates a fundamental challenge—Chinese word boundaries are determined by semantics and do not always permit arbitrary decomposition.

### 5 Evaluation Metrics

Standard MT metrics such as BLEU measure n-gram overlap and do not capture whether a translation preserves musical structure. Kim et al. (2023) proposed a computational evaluation framework for singable lyric translation; building on this direction, we propose three complementary metrics, each targeting one of the three musical constraints.

#### 5.1 Syllable Error Rate (SER)

SER measures the overall alignment between the source and translated syllable count sequences using the Levenshtein edit distance (Levenshtein, 1966):

$$\text{SER} = \frac{\text{Lev}(\mathbf{s}, \mathbf{s}')}{\max(|\mathbf{s}|, |\mathbf{s}'|)} \quad (3)$$

where  $\mathbf{s} = (s_1, \dots, s_n)$  and  $\mathbf{s}' = (s'_1, \dots, s'_m)$  are the source and translated syllable count sequences, and Lev operates over sequences of integers (with substitution cost equal to 1 regardless of the magnitude of the difference).

SER captures both *substitution errors* (a line has the wrong syllable count) and *structural errors* (lines are missing or added). SER = 0 indicates

perfect preservation; higher values indicate greater divergence. Unlike per-line metrics, SER penalizes translations that drop or add lines, which is important because line structure directly corresponds to melodic phrases.

## 5.2 Syllable Count Relative Error (SCRE)

SCRE provides a normalized, per-line view of syllable accuracy, assuming the translation preserves the number of lines ( $m = n$ ):

$$\text{SCRE} = \frac{1}{n} \sum_{i=1}^n \frac{|s_i - s'_i|}{s_i} \quad (4)$$

SCRE weights errors relative to line length: a two-syllable error on a four-syllable line ( $\text{SCRE}_i = 0.5$ ) is penalized more heavily than on a twelve-syllable line ( $\text{SCRE}_i = 0.17$ ). This reflects the intuition that short lines leave less room for syllabic deviation—a single extra syllable in a three-syllable phrase is far more noticeable than in a long verse.  $\text{SCRE} = 0$  indicates all lines match exactly.

SER and SCRE are complementary: SER captures structural alignment including line count mismatches, while SCRE provides percentage-based accuracy that is more interpretable for line-by-line comparison.

## 5.3 Adjusted Rand Index (ARI)

To measure rhyme scheme preservation, we treat the rhyme scheme as a *clustering* of lines and compute the Adjusted Rand Index (Hubert and Arabie, 1985) between the source and translated clusterings.

Let  $R_s = (r_1, \dots, r_n)$  and  $R_t = (r'_1, \dots, r'_n)$  be the rhyme scheme labelings. The Rand Index (RI) counts the fraction of line pairs that are either in the same cluster in both schemes or in different clusters in both schemes. ARI adjusts RI for the expected value under random label assignments:

$$\text{ARI} = \frac{\text{RI} - \mathbb{E}[\text{RI}]}{\max(\text{RI}) - \mathbb{E}[\text{RI}]} \quad (5)$$

$\text{ARI} \in [-1, 1]$ : 1 indicates the translation perfectly preserves the rhyme structure, 0 indicates chance-level agreement, and negative values indicate systematic disagreement. Crucially, ARI is robust to label permutation—a source with scheme *AABB* and a translation with *BBAA* receives  $\text{ARI} = 1$ , since the grouping structure is identical.

Together, SER, SCRE, and ARI cover the structural dimensions of singable translation. They deliberately do not measure semantic fidelity or naturalness; we view them as complementary to standard MT metrics such as BLEU and COMET. A good singable translation should score well on both semantic quality metrics and structural quality metrics—high BLEU with high SER would indicate a good translation that is unsingable, while low SER with low BLEU would be structurally correct but semantically wrong.

## 6 Experiments

We conduct preliminary experiments to evaluate the effectiveness of IPA-augmented tool use for constraint-preserving lyrics translation.

### 6.1 Setup

We evaluate on English–Chinese song translation in both directions: English-to-Chinese ( $\text{en} \rightarrow \text{cmn}$ ) and Chinese-to-English ( $\text{cmn} \rightarrow \text{en}$ ). Our test set consists of 500 popular songs scraped from a commercial music platform, spanning pop, ballad, and rock genres from the 2000s to 2020s, with roughly equal coverage of English and Chinese tracks. Each test case uses 4–20 lines from a song, with lines filtered to contain only the source language via automatic language detection. Due to copyright restrictions, we do not distribute the lyrics themselves; our open-source release (Apache-2.0) includes the full translation framework, IPA tool suite, LangGraph agent definitions, and evaluation scripts as a PyPI package (`blt-toolkit`), enabling reproduction on any user-supplied lyrics.

**Implementation.** The agent framework is implemented using LangGraph, which provides a graph-based orchestration layer for defining the multi-phase agent loop described in §4.4. Each phase is represented as a node in a directed graph with conditional edges encoding the control flow between constraint extraction, initial translation, syllable refinement, and pattern refinement. For LLM inference, we use Qwen3-30B (4-bit quantized) served locally via Ollama, which provides an OpenAI-compatible API for function calling. The IPA tools (§4.2) are registered as callable functions through LangGraph’s tool-binding interface, enabling the ReAct-style reasoning-and-acting loop.

**Conditions.** We compare two conditions using the same underlying model:

| Method    | SER ↓         | SCRE ↓        |
|-----------|---------------|---------------|
| Base LLM  | 0.8729        | 1.2756        |
| BLT Agent | <b>0.5970</b> | <b>0.4499</b> |

Table 3: Overall results aggregated across both translation directions. Lower is better for both metrics. Bold indicates the best result.

- **Base LLM:** Direct translation via Qwen3-30B with a prompt requesting syllable-count preservation, but *without* access to IPA tools or iterative refinement.
- **BLT Agent:** Our full framework with IPA tool suite and multi-phase refinement loop (Phases 1–3 as described in §4.4), orchestrated by LangGraph.

Both conditions use the same model, isolating the effect of tool-augmented iterative refinement. We report SER and SCRE as defined in §5. We do not report ARI because our 4–20-line excerpts frequently contain only one or two rhyme classes, making the clustering comparison degenerate: when most lines fall into a single class, ARI is dominated by chance agreement and becomes unstable. A meaningful ARI evaluation requires excerpts with at least two rhyme classes of two or more lines each; we estimate fewer than 30% of our test cases meet this criterion, insufficient for reliable aggregate statistics.

## 6.2 Overall Results

Table 3 presents the aggregate results across both translation directions.

The BLT Agent substantially outperforms the Base LLM on both metrics. SER decreases from 0.8729 to 0.5970 (31.6% relative reduction), indicating that the tool-augmented agent produces syllable count sequences much closer to the source. The improvement on SCRE is even more pronounced: a reduction from 1.2756 to 0.4499 (64.7% relative reduction). A Base LLM SCRE of 1.2756 means that, on average, each line deviates from the target syllable count by more than 100% relative to the target—effectively, the baseline pays little attention to syllable constraints. The BLT Agent reduces this to 0.4499, still imperfect but representing a meaningful step toward singable translation.

| Pair     | SER ↓        |       | SCRE ↓       |              |
|----------|--------------|-------|--------------|--------------|
|          | Agent        | Base  | Agent        | Base         |
| en → cmn | <b>0.238</b> | 0.810 | <b>0.085</b> | 0.250        |
| cmn → en | <b>0.577</b> | 0.811 | 0.258        | <b>0.246</b> |

Table 4: Per-language-pair results. Bold indicates the best result for each metric and direction.

## 6.3 Per-Language-Pair Results

Table 4 breaks down performance by translation direction, revealing an asymmetry between the two directions.

**English-to-Chinese.** The en → cmn direction shows the strongest improvements. SER drops from 0.810 to 0.238 (70.6% relative improvement) and SCRE from 0.250 to 0.085 (66.0%). An SCRE of 0.085 means the average line deviates by only 8.5% from the target syllable count—close to singable quality. We attribute this strong performance to the fact that Chinese syllable counting is perfectly deterministic (one character = one syllable), making the COUNTSYLLABLES tool maximally reliable for verification in this direction.

**Chinese-to-English.** The cmn → en direction proves more challenging. While the Agent still improves SER substantially (0.577 vs. 0.811, a 28.9% reduction), SCRE is slightly worse than the baseline (0.258 vs. 0.246). This asymmetry likely reflects the greater difficulty of English syllable counting: IPA-based vowel nucleus detection, while generally accurate, can miscount in cases involving schwa reduction, syllabic consonants, or diphthong ambiguities. Furthermore, English morphology affords less flexibility in adjusting syllable counts—adding or removing a syllable often requires restructuring the entire phrase, whereas Chinese can often swap individual characters.

## 6.4 Qualitative Analysis

Table 5 illustrates the pipeline on the *Let It Go* excerpt from Table 2, translated from English to Chinese. The Base LLM produces natural but unconstrained translations: line counts range from 7 to 9 characters, overshooting every target. The BLT Agent, guided by COUNTSYLLABLES verification in Phase 2, matches three of four targets exactly. Line 4 (target 8) remains one syllable short at 7—Phase 2 exhausted its 10 refinement attempts without finding an 8-character alternative that preserved

| Line                                        | Text                       | Syll. | Tgt |
|---------------------------------------------|----------------------------|-------|-----|
| <b>Source (English):</b>                    |                            |       |     |
| 1                                           | The snow glows white       | 4     | —   |
| 2                                           | On the mountain tonight    | 6     | —   |
| 3                                           | Not a footprint to be seen | 7     | —   |
| 4                                           | A kingdom of isolation     | 8     | —   |
| <b>Base LLM (Chinese, no tools):</b>        |                            |       |     |
| 1                                           | 白雪在夜裡閃耀                    | 7     | 4 × |
| 2                                           | 今夜在那座山上                    | 7     | 6 × |
| 3                                           | 看不見任何一個腳印                  | 9     | 7 × |
| 4                                           | 一個與世隔絕的王國                  | 9     | 8 × |
| <b>BLT Agent (Chinese, with IPA tools):</b> |                            |       |     |
| 1                                           | 皚皚白雪                       | 4     | 4 ✓ |
| 2                                           | 籠罩今夜山巔                     | 6     | 6 ✓ |
| 3                                           | 看不到一絲足跡                    | 7     | 7 ✓ |
| 4                                           | 這是孤獨的國度                    | 7     | 8 × |

Table 5: Qualitative example: English-to-Chinese translation of the opening lines of *Let It Go*. “Syll.” is the actual character count; “Tgt” is the source syllable count that the translation should match. The BLT Agent matches 3/4 targets; the Base LLM matches 0/4.

meaning, retaining the closest attempt. Phase 3 pattern refinement has limited effect here because Chinese character-level tokenization produces uniform  $[1, 1, \dots]$  patterns regardless of word grouping.

## 6.5 Discussion and Future Directions

These results demonstrate that tool-augmented agents provide substantial benefits for constraint-preserving lyrics translation, particularly when the target language has simple, deterministic syllable structure. However, several important directions remain.

**Comparison with prior systems.** Our evaluation compares only against a base LLM without tool access. Prior constrained translation systems—including the fine-tuned models of [Ou et al. \(2023a\)](#) and [Guo et al. \(2022\)](#), and the RL-based approach of [Ren et al. \(2025\)](#)—represent stronger baselines. We did not reproduce these systems because they require task-specific training pipelines and parallel lyric corpora that are not publicly available. Future work should evaluate our metrics on outputs from these systems to enable direct comparison; the MAVL benchmark ([Cho et al., 2025](#)) may facilitate this.

**SCORE degradation in cmn→en.** The agent’s SCORE is slightly worse than the baseline for Chinese-to-English (0.258 vs. 0.246), even though SER improves. We attribute this to two factors: (1) the iterative refinement sometimes overshoots, pro-

ducing lines that are further from the target than the baseline’s initial attempt, and (2) English syllable counting via IPA vowel nucleus detection introduces noise from schwa reduction and diphthong ambiguity, causing the agent to optimize toward an incorrect target. An intrinsic evaluation of tool accuracy—measuring syllable-counting precision against human annotations—would help quantify this effect and is a priority for future work.

**Rhyme preservation.** Our framework monitors rhyme via DETECTRHYME but lacks a dedicated refinement phase for it. ARI is theoretically well-suited for rhyme evaluation but is unstable on our short excerpts: with only 4–20 lines, most excerpts have sparse rhyme structure (often a single dominant class), and ARI degenerates. Future work should either (1) evaluate on full songs with richer rhyme patterns, restricting ARI reporting to excerpts with at least two rhyme classes of two or more lines each, or (2) adopt a lighter-weight proxy such as pairwise rhyme precision/recall on line pairs that share a rhyme label in the source. Adding a dedicated rhyme refinement phase is also a natural next step.

**Broader language coverage and model variation.** Our evaluation is limited to English–Chinese with a single LLM (Qwen3-30B). Extending to typologically diverse language pairs and multiple model sizes would strengthen generalizability claims.

**Human evaluation.** Our automatic metrics measure structural constraint preservation but not semantic fidelity, naturalness, or subjective singability. A comprehensive evaluation should include human judgments from bilingual speakers and singers, which we leave to future work.

## 7 Conclusion

We have presented a framework for lyrics translation that preserves musical constraints through tool-augmented LLM agents. We formalized three musical constraints—syllable counts, rhyme schemes, and syllable patterns—and described IPA-based methods for their extraction and verification. By equipping an LLM with phonetic analysis tools and structuring translation as a multi-phase ReAct loop, our system iteratively refines translations to satisfy these constraints in priority order: syllable counts first, then patterns, with rhyme monitored throughout. The proposed metrics—SER, SCORE, and ARI—provide automatic measurement



of constraint preservation that complements standard translation quality evaluation.

Preliminary experiments on English–Chinese translation demonstrate that tool-augmented agents substantially reduce syllable error rates compared to direct LLM translation, with particularly strong results in the English-to-Chinese direction where syllable counting is deterministic. The tool-augmented agent approach has a key advantage over constrained decoding methods (Hokamp and Liu, 2017; Post and Vilar, 2018): it is model-agnostic and does not require modifying the decoding algorithm. Any LLM with function-calling capabilities can serve as the agent, and the IPA tools can be extended to support additional constraints (e.g., stress patterns, tonal matching) without changing the agent architecture.

Future work will focus on evaluation across diverse language pairs and LLM architectures, human judgments of singability, and integration with singing voice synthesis systems (Ren et al., 2020) for end-to-end singable translation.

## Limitations

Our approach has several limitations. First, the framework’s correctness depends on the accuracy of the underlying phonetic tools. Phonemizer relies on eSpeak NG for IPA transcription, which has uneven quality across languages; we do not provide an intrinsic evaluation of syllable-counting or rhyme-detection accuracy even for English and Chinese. Errors in tool outputs can cause the agent to refine toward incorrect targets, as likely occurs in the cmn→en SCRE degradation. Second, rhyme preservation is monitored but not actively refined: the agent lacks a dedicated rhyme refinement phase, and we do not report quantitative rhyme results (ARI) due to the short excerpt lengths in our test set. Third, the syllable pattern constraint is inherently harder to enforce for language pairs with very different morphological structures—Chinese words are typically 1–2 syllables while English words are longer, making exact pattern matching often impossible. Fourth, our evaluation compares only against a base LLM; we do not reproduce prior constrained translation systems (Ou et al., 2023a; Guo et al., 2022; Ren et al., 2025) as baselines, limiting our ability to claim progress over the state of the art. Fifth, the multi-phase refinement loop introduces latency: each phase involves multiple LLM inference calls with tool interactions, and translating

a single verse may require dozens of tool calls, though we do not measure runtime or cost. Sixth, our metrics measure structural constraint preservation but not semantic fidelity or naturalness; a complete evaluation requires human judgments. Finally, while the technical approach—calling deterministic phonetic tools inside a ReAct loop—applies established patterns (Yao et al., 2023; Gao et al., 2023; Schick et al., 2023) to a new domain, the primary contribution is the formalization of musical constraints and the IPA-based tool suite rather than a novel agent architecture.

## References

- Mathieu Bernard and Hadrien Titeux. 2021. **Phonemizer: Text to phones transcription for multiple languages in Python**. *Journal of Open Source Software*, 6(68):3958.
- Yiwen Chen and Simone Teufel. 2024. Scansion-based lyrics generation. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 14370–14381.
- Woohyun Cho, Youngmin Kim, Sunghyun Lee, and Youngjae Yu. 2025. MAVL: A multilingual audio-video lyrics dataset for animated song translation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*.
- Johan Franzon. 2008. **Choices in song translation: Singability in print, subtitles and sung performance**. *The Translator*, 14(2):373–399.
- Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. **PAL: Program-aided language models**. In *Proceedings of the 40th International Conference on Machine Learning*, pages 10764–10799.
- Marjan Ghazvininejad, Yejin Choi, and Kevin Knight. 2018. Neural poetry translation. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*.
- Marjan Ghazvininejad, Xing Shi, Yejin Choi, and Kevin Knight. 2016. **Generating topical poetry**. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 1183–1191.
- Fenfei Guo, Chen Zhang, Zhirui Zhang, Qixin He, Kecheng Zhang, Jun Xie, and Jordan Boyd-Graber. 2022. Automatic song translation for tonal languages. In *Findings of the Association for Computational Linguistics: ACL 2022*.



872 Songyan Zhao, Bingxuan Li, Yufei Tian, and Nanyun  
873 Peng. 2025. REFFLY: Melody-constrained lyrics  
874 editing model. In *Proceedings of the 2025 Confer-*  
875 *ence of the Nations of the Americas Chapter of the*  
876 *Association for Computational Linguistics: Human*  
877 *Language Technologies (Volume 1: Long Papers)*,  
878 pages 11295–11315.